

# Guía Práctica N.º 07

## Cliente Ktor y consumo GET

Entregables — Unidad 2, Sesión 1

---

|                         |                                                                                                           |
|-------------------------|-----------------------------------------------------------------------------------------------------------|
| <b>Estudiante:</b>      | Jhonsons Plasencia Valdez                                                                                 |
| <b>Curso:</b>           | Desarrollo de Aplicaciones Móviles                                                                        |
| <b>Universidad:</b>     | Universidad Peruana Unión (UPeU)                                                                          |
| <b>Docente:</b>         | Reyna Barreto Benjamin David                                                                              |
| <b>Proyecto:</b>        | PharmaMobile (Kotlin Multiplatform)                                                                       |
| <b>Rama de trabajo:</b> | feature/ktor-client                                                                                       |
| <b>Repositorio:</b>     | <a href="https://github.com/Plasencia2007/pharmamobile">https://github.com/Plasencia2007/pharmamobile</a> |

## 1. Backend consumido

Esta práctica originalmente apuntaba a la API pública de entrenamiento (escuelajs), pero durante el desarrollo se migró el cliente Ktor para consumir el backend real del curso, **PharmaSoft** (<https://github.com/dreyna/pharmaSoft>), un servicio Spring Boot con persistencia en PostgreSQL. El repositorio del backend se portó de Oracle a PostgreSQL (driver `org.postgresql`, migración Flyway reescrita) al no contar con una instancia Oracle disponible en la máquina de desarrollo.

|                              |                                                                                                                     |
|------------------------------|---------------------------------------------------------------------------------------------------------------------|
| URL base (Android, emulador) | http://10.0.2.2:8082/api/v1/                                                                                        |
| URL base (iOS, simulador)    | http://localhost:8082/api/v1/                                                                                       |
| Endpoint consumido           | GET /productos?pagina=0&tamano=20 (paginado)                                                                        |
| Wrapper de paginación        | PaginaResponseDTO → contenido, pagina, tamano, totalElementos, totalPaginas, ultima                                 |
| Campos de ProductoDto        | id: Long, nombre: String, precio: Double, stock: Int, estado: Boolean, categoriaId: Long?, categoriaNombre: String? |
| Mapeo a dominio              | estado → Producto.activo (resto 1:1)                                                                                |

El backend PharmaSoft expone un CRUD completo (productos, categorías, clientes, ventas, reportes). Esta sesión implementa únicamente el **GET** de productos, que es el alcance de la guía; el registro desde el formulario (POST) sigue guardando localmente en memoria hasta que se conecte en una sesión posterior.

## 2. Repositorio, rama y commits

Repositorio: <https://github.com/Plasencia2007/pharmamobile>

Rama: `feature/ktor-client` (publicada en el remoto)

Último commit: 245e6f8

| Commit  | Mensaje                                                                    |
|---------|----------------------------------------------------------------------------|
| b69d273 | feat: agrega dependencias de Ktor 3.6.0 y plugin de serializacion          |
| e6dbe6d | feat: declara el motor HTTP por plataforma (OkHttp/Darwin)                 |
| df7cc76 | feat: agrega ProductoDto/CategoriaDto y mapper hacia el dominio            |
| 05ba525 | feat: crea HttpClientFactory con ContentNegotiation, Logging y HttpTimeout |
| 372d416 | feat: conecta ProductoApi y ProductoRepositoryRemoto via Koin              |
| 245e6f8 | feat: conecta el cliente Ktor al backend real PharmaSoft (no a escuelaJs)  |

### 3. Evidencia — Log de Ktor (GET + respuesta 200)

Captura de adb logcat filtrada por el proceso de PharmaMobile, mostrando la petición GET hacia el backend PharmaSoft y la respuesta 200 OK.

```
adb logcat -- pe.edu.upeu.pharmamobile (System.out)
09-30 19:45:29.228 8352 8352 I System.out: HttpClient: REQUEST: http://10.0.2.2:8082/api/v1/productos?pagina=0&tamano=20
09-30 19:45:29.228 8352 8352 I System.out: METHOD: GET
09-30 19:45:29.228 8352 8352 I System.out: COMMON HEADERS
09-30 19:45:29.229 8352 8352 I System.out: -> Accept: application/json
09-30 19:45:29.229 8352 8352 I System.out: -> Content-Type: application/json
09-30 19:45:29.229 8352 8352 I System.out: CONTENT HEADERS
09-30 19:45:29.229 8352 8352 I System.out: -> Content-Length: 0
09-30 19:45:30.003 8352 8352 I System.out: HttpClient: RESPONSE: 200
09-30 19:45:30.003 8352 8352 I System.out: METHOD: GET
09-30 19:45:30.003 8352 8352 I System.out: FROM: http://10.0.2.2:8082/api/v1/productos?pagina=0&tamano=20
09-30 19:45:30.003 8352 8352 I System.out: COMMON HEADERS
09-30 19:45:30.003 8352 8352 I System.out: -> connection: keep-alive
09-30 19:45:30.003 8352 8352 I System.out: -> content-type: application/json
09-30 19:45:30.003 8352 8352 I System.out: -> date: Wed, 30 Sep 2026 19:45:30 GMT
09-30 19:45:30.003 8352 8352 I System.out: -> keep-alive: timeout=60
09-30 19:45:30.003 8352 8352 I System.out: -> transfer-encoding: chunked
09-30 19:45:30.003 8352 8352 I System.out: -> vary: Origin; Access-Control-Request-Method; Access-Control-Request-Headers
```

Figura 1. Log HTTP del cliente Ktor: REQUEST GET → RESPONSE 200.

### 4. Evidencia — Lista de productos en Android (emulador)

Pantalla "Productos" de PharmaMobile mostrando el inventario obtenido en tiempo real desde el backend PharmaSoft (datos creados vía API para la demostración: Paracetamol 500mg, Ibuprofeno 400mg).

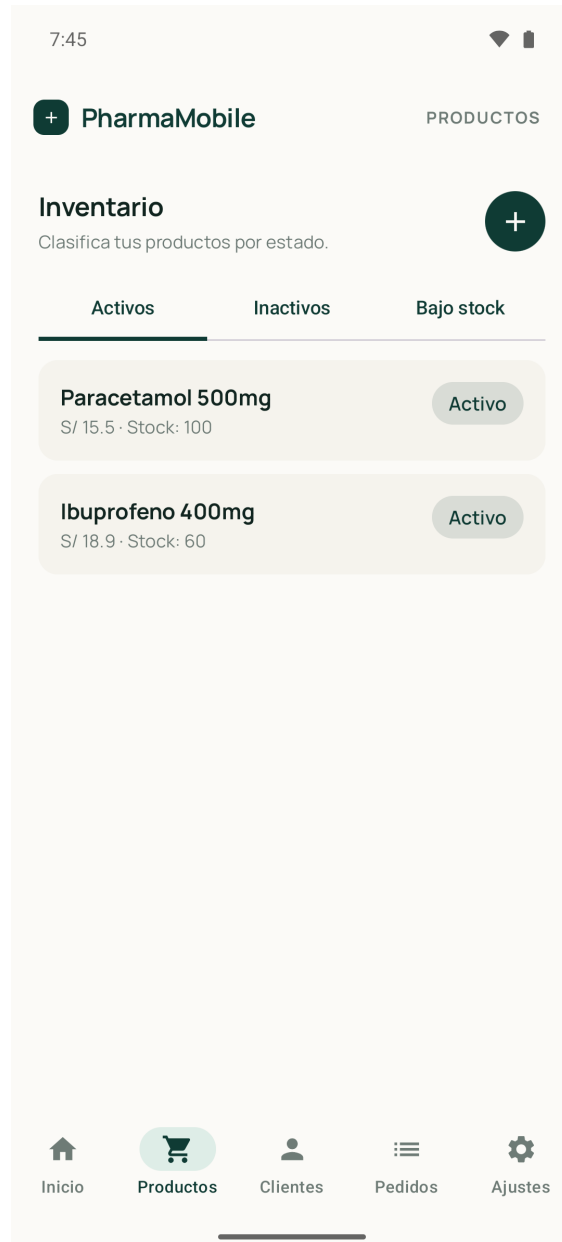


Figura 2. Inventario renderizado con datos reales del backend (Android).

## 5. Verificación en iOS (simulador)

No se cuenta con una máquina macOS en este entorno de desarrollo, por lo que no fue posible ejecutar el simulador de iOS ni capturar una pantalla equivalente. La integración se validó en su lugar mediante la compilación del target de iOS:

```
> ./gradlew :shared:compileKotlinIosSimulatorArm64  
BUILD SUCCESSFUL
```

Esto confirma que HttpClientFactory, ProductoApi, ProductoRepositoryRemoto y el motor Darwin (declarado en PlatformModule.ios.kt) compilan correctamente para iOS, incluyendo la constante urlBaseApi específica de esa plataforma (<http://localhost:8082/api/v1/>).

**Pendiente:** capturar la pantalla en un simulador real de iOS cuando se disponga de macOS/Xcode.

## 6. Evidencia — Manejo de error sin conectividad

Con el emulador sin WiFi ni datos móviles (equivalente a modo avión), la aplicación relanzada muestra un mensaje de error legible en el estado "Error" de la UI, sin crashear.

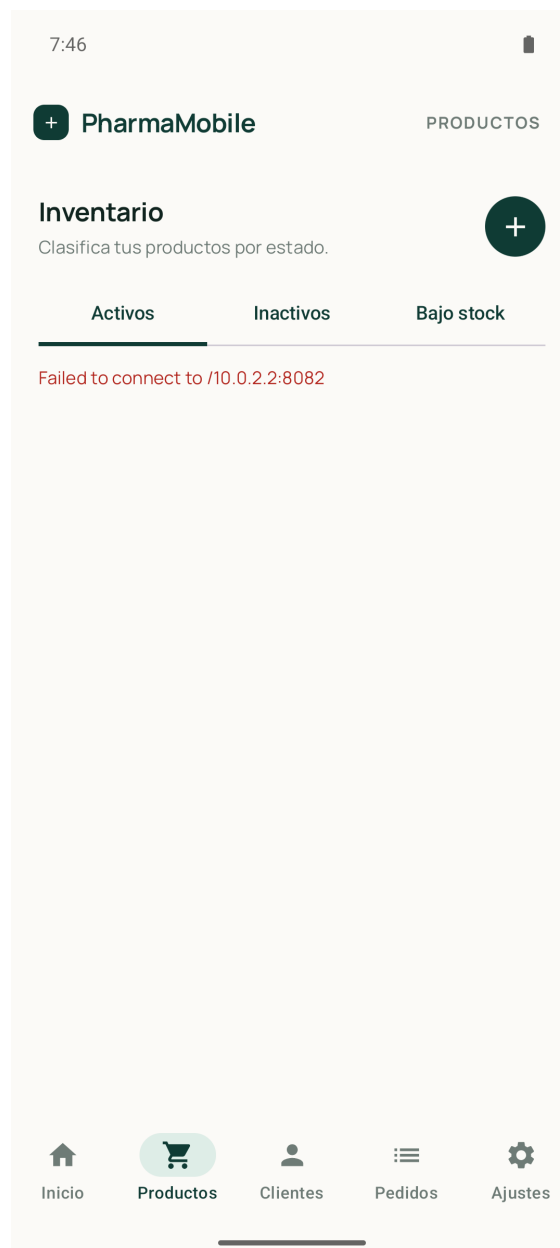


Figura 3. "Failed to connect to /10.0.2.2:8082" — error capturado por runCatching en ProductoViewModel y mostrado en FaseInventario.Error, sin crash de la app.

## 7. Autoevaluación contra la rúbrica (20 pts)

| Criterio                                                         | Pts | Estado                                               |
|------------------------------------------------------------------|-----|------------------------------------------------------|
| Dependencias de Ktor declaradas por source set                   | 2   | Cumplido                                             |
| Permiso INTERNET + tráfico HTTPS/cleartext correcto              | 2   | Cumplido                                             |
| Motores OkHttp/Darwin declarados correctamente                   | 2   | Cumplido                                             |
| HttpClient creado una sola vez e inyectado via Koin              | 2   | Cumplido                                             |
| ContentNegotiation/Logging/HttpTimeout instalados y configurados | 2   | Cumplido                                             |
| DTOs anotados con @Serializable y valores por defecto sensatos   | 2   | Cumplido                                             |
| Mapper existe y el dominio no conoce la red                      | 2   | Cumplido                                             |
| GET retorna 200 y la lista se renderiza en Compose               | 2   | Cumplido                                             |
| App corre en Android e iOS con evidencia gráfica                 | 2   | Parcial (solo Android; iOS validado por compilación) |
| Error de red manejado y comunicado sin crashear                  | 2   | Cumplido                                             |

Puntaje estimado: 19 / 20 (criterio de iOS parcial por falta de hardware Mac).

## 8. README actualizado

El archivo README.md del proyecto incluye ahora la sección "Consumo de API (Unidad 2, Sesión 1 — Cliente Ktor)" con: instrucciones para levantar el backend PharmaSoft localmente (Docker + PostgreSQL + mvnw spring-boot:run), la URL base por plataforma, el endpoint consumido y los campos del DTO utilizados. Ver README .md en la raíz del repositorio.